Fact checking binario con evidenza evidenziata — panoramica

Questa cartella estende ../fine_tuning_factcheck/. Il compito resta binario (documento + claim → supportato / non supportato), con un'uscita in più:

> **se il claim è supportato**, il modello indica le frasi del documento che lo stabiliscono.

Non si evidenzia nulla nei casi negativi: né le frasi che contraddicono il claim, né le parti del claim che mancano nel documento. Se il verdetto è "non supportato", l'uscita è solo il verdetto.

Vincolo di progetto: **la velocità deve restare quella di un decision model**, cioè un solo forward sul prompt e nessun token generato.

1. Il problema in una pagina

Input: documento e claim (il documento sempre fornito, nessun retrieval).

Output:

```
{"p_supported": 0.93, "supported": true,
 "evidence": [{"sentence": 0,  "start": 0,    "end": 61,   "score": 0.91},
              {"sentence": 39, "start": 4210, "end": 4262, "score": 0.84}]}
```

Se `supported` è `false`, `evidence` è vuoto.

Esempio con evidenza sparsa (il caso difficile):

> Claim: "Nancy R. Howell teaches at a seminary in Kansas City."
>
> S1 (inizio del documento): "Saint Paul School of Theology is a seminary located in Kansas City, Missouri." ... 38 frasi irrilevanti ... S40 (fine): "Nancy R. Howell is Professor of Theology at Saint Paul School of Theology."
>
> Verdetto: supportato. Evidenza: S1 e S40.

2. L'architettura scelta (in breve)

Un solo forward del decision model fine-tunato, con il **claim prima del documento** nel prompt. Dallo stesso forward si leggono due cose:

```
prompt: [istruzioni] [claim] [documento: S1 … Sn] [domanda] Answer:
                          |                            |
              forward unico del LLM (LoRA)             |
                          |                            |
  h_1 … h_n = stati all'ultimo token di ogni frase    q = stato in "Answer:"
                          |                            |
                          |            logit delle lettere → P(supportato)   (come oggi)
                          ▼
  aggregatore: proiezione d→256 + 2 layer di transformer NON causale su [q, h_1 … h_n]
                          |
          Linear → P(frase k è evidenza)   → mostrata solo se P(supportato) > soglia
```

Perché ogni pezzo (dettagli e alternative scartate in 00a_scelta_approccio.md):

Scelta	Motivo
Un forward, nessuna generazione	ogni token generato costa ~25 ms per un 12B in bf16 su L40S: le citazioni generate raddoppierebbero la latenza
Claim prima del documento	con l'attenzione causale, le frasi vedono solo ciò che le precede: devono vedere il claim
Stati all'ultimo token di ogni frase	nessun marcatore nel testo: il documento resta identico, le posizioni si ricavano dagli offset del tokenizer

Scelta	Motivo
Aggregatore bidirezionale	la prima frase non vede l'ultima; l'aggregatore fa comunicare le frasi fra loro e con `q`, che ha visto tutto. Risolve l'evidenza sparsa
Testa solo sui positivi	il compito è evidenziare solo quando l'evidenza c'è

Costo aggiuntivo dell'aggregatore: qualche milione di operazioni su n vettori da 256, contro decine di TFLOP del forward: trascurabile.

3. Il percorso

Stessa struttura delle altre due cartelle.

Step	File	Contenuto
0. Split	00b_split_e_protocollo.md	split del piano binario + sottoinsiemi con evidenza annotata (dev e test)
0a. Approccio	00a_scelta_approccio.md	perché una testa nello stesso forward; il problema dell'evidenza sparsa
1. Baseline	01_baseline.md	verdetto (dal piano binario) + evidenza senza training (somiglianza, attenzione, ablazione offline)
2a. Fonti	02a_dataset_fonti_esistenti.md	FEVER, HoVer, SciFact, WiCE: evidenza a livello di frase e come allinearla ai documenti
2b. Generazione	02b_dataset_generazione_llm.md	C2D/CG2C con posizioni note, etichettatura per ablazione offline, evidenza sparsa costruita
2c. Replay	02c_replay_e_forgetting.md	non serve; il rischio è che la testa peggiori il verdetto
2d. Varianti	02d_varianti_domanda.md	prompt fisso, claim prima del documento
2e. Mix	02e_mix_finale.md	quota di esempi con evidenza, maschere, augmentation
3. Rendering	03_rendering_formato.md	segmentazione in frasi, posizioni dei token di fine frase, formato
4. Loss	04_loss.md	CE del verdetto + BCE per frase mascherata + ranking
5. Training	05_training.md	aggregatore, LoRA, backbone congelato o no
6. Valutazione	06_valutazione.md	F1 per frase, recall dell'insieme completo, distanza, latenza
7. Esperimenti	07_esperimenti.md	piano H (evidenza)
8. Export	08_merge_export.md	pesi + testa, server che restituisce stati e logit dallo stesso forward
Rischi	09_rischi_checklist.md	

4. Le decisioni chiave

1. **Si parte dal piano binario.** Verdetto, dati, loss sulle lettere e protocollo sono quelli di ../fine_tuning_factcheck/; qui si aggiunge l'evidenza.

2. **Claim prima del documento in tutto il piano**, verdetto compreso: il formato deve essere lo stesso in training e in produzione.
3. **Prima il verdetto, poi l'evidenza.** Si addestra il verdetto (piano binario, con il claim prima), poi la testa di evidenza: prima su backbone congelato (il verdetto non cambia di un bit), poi, se serve, insieme alla LoRA.
4. **Evidenza solo dove il dato è affidabile.** La loss per frase si applica solo agli esempi supportati con evidenza nota; tutto il resto è mascherato.
5. **Evidenza sparsa trattata esplicitamente**: aggregatore bidirezionale, documento intero in un forward (niente chunking con `max`), dati con evidenza messa apposta lontana.
6. **Velocità misurata**: la latenza con testa deve essere entro +3% di quella del solo verdetto.

5. Obiettivi misurabili

Obiettivo	Soglia
Verdetto	nessun calo rispetto al modello del piano binario (BAcc e AUC su `dev_select`, entro 0,3 punti)
F1 per frase sull'evidenza (dev con evidenza)	$>$ baseline per somiglianza di almeno 15 punti; obiettivo indicativo $\geq$ 0,75 su FEVER/HoVer
Recall dell'insieme completo (tutte le frasi gold evidenziate)	$\geq$ 0,65 su HoVer (multi-hop)
Calo con la distanza fra frasi di evidenza	$<$ 5 punti di F1 fra "adiacenti" e "oltre 20 frasi"
Sufficienza (le frasi evidenziate da sole danno "supportato")	$\geq$ 85% dei positivi veri
Latenza	$\leq$ +3% rispetto al solo verdetto

6. Tempi (stime)

Fase	Tempo
Builder con evidenza (FEVER, HoVer, SciFact, WiCE)	1–2 giorni di codice
Etichettatura per ablazione offline e generazione con posizioni	1–2 giorni di GPU
Testa su backbone congelato (estrazione degli stati + training)	poche ore
Training congiunto LoRA + testa	come una corsa del piano binario (+10%)